

Functional Dependencies, Redundancy and Data Anomalies

1. Objective

The objective of Week 4 is to examine the database design prepared in Weeks 1 to 3 and to:

1. Identify the functional dependencies (FDs) between the attributes of the E-Commerce system.
2. Analyse where data is unnecessarily repeated (data redundancy).
3. Understand how poor database design leads to insertion, deletion and update anomalies.

Notation used: $A \rightarrow B$ means that A functionally determines B, that is, for every value of A there is exactly one value of B. A is called the *determinant*. For example, $\text{Customer_ID} \rightarrow \text{Name}$ means a given Customer_ID always has one name.

2. Summary of Work Done in Weeks 1 to 3

2.1 Week 1: Business Problem and Requirements

The proposed system supports an online shopping platform similar to Amazon or Flipkart. The main activities identified were:

- Customer registration and profile management
- Product and category management
- Inventory tracking
- Order placement and order history
- Payment processing
- Shipment tracking
- Product reviews and ratings
- Sales reporting and business analytics

2.2 Week 2: Entities, Attributes and Primary Keys

Entity	Primary key	Other attributes
Customer	Customer_ID	Name, Email, Mobile, Address, Registration_Date
Category	Category_ID	Category_Name, Description
Product	Product_ID	Product_Name, Price, Stock_Quantity, Category_ID
Supplier	Supplier_ID	Supplier_Name, Contact_Info
Orders	Order_ID	Customer_ID, Order_Date, Total_Amount, Order_Status
Order_Details	Order_Detail_ID	Order_ID, Product_ID, Quantity,

Entity	Primary key	Other attributes
		Unit_Price
Payment	Payment_ID	Order_ID, Payment_Method, Payment_Date, Payment_Status
Shipment	Shipment_ID	Order_ID, Delivery_Address, Shipment_Date, Delivery_Status
Review	Review_ID	Customer_ID, Product_ID, Rating, Comments

2.3 Week 3: Relationships and Relational Schema

Relationship	Type
Customer places Orders	One-to-Many
Category contains Products	One-to-Many
Orders contain Order_Details	One-to-Many
Product appears in Order_Details	One-to-Many
Orders and Products	Many-to-Many (resolved by Order_Details)
Order has Payment	One-to-One
Order has Shipment	One-to-One
Customer writes Reviews	One-to-Many
Product receives Reviews	One-to-Many

Foreign keys in the relational schema: Product.Category_ID, Orders.Customer_ID, Order_Details.Order_ID, Order_Details.Product_ID, Payment.Order_ID, Shipment.Order_ID, Review.Customer_ID, Review.Product_ID.

In Week 4 this design is reviewed for dependencies, redundancy and anomalies.

3. Key Terms

Term	Meaning
Functional dependency	A relationship $A \rightarrow B$ where A uniquely determines B

Term	Meaning
Full dependency	A non-key attribute depends on the whole composite primary key
Partial dependency	A non-key attribute depends on only part of a composite primary key
Transitive dependency	A non-key attribute depends on another non-key attribute ($A \rightarrow B$ and $B \rightarrow C$, so $A \rightarrow C$)
Candidate key	Any attribute or set of attributes that can uniquely identify a row
Data redundancy	The same fact stored in more than one place
Anomaly	An error or inconsistency caused by redundancy during insert, delete or update

4. Why Poor Design Is Studied: A Single-Table (Flat) Design

To see how bad design causes problems, imagine the whole business stored in one table, as a beginner might first sketch it.

ORDER_FLAT (Order_ID, Order_Date, Order_Status, Customer_ID, Cust_Name, Cust_Email, Cust_Mobile, Cust_Address, Product_ID, Product_Name, Price, Category_ID, Category_Name, Supplier_ID, Supplier_Name, Quantity, Unit_Price, Payment_Method, Payment_Status, Shipment_Status)

Primary key: (Order_ID, Product_ID), because one order can contain many products, and the same product can appear in many orders.

4.1 Sample Data (selected columns)

Order_ID	Customer_ID	Cust_Name	Cust_Address	Product_ID	Product_Name	Price	Category_Name	Qty
1001	C01	Arun	12 Anna Nagar, Trichy	P01	Laptop	55000	Electronics	1
1001	C01	Arun	12 Anna Nagar, Trichy	P02	Mouse	500	Electronics	2
1002	C01	Arun	12 Anna Nagar, Trichy	P02	Mouse	500	Electronics	1
1003	C02	Divya	4 Gandhi Rd,	P01	Laptop	55000	Electronics	1

Order_ID	Customer_ID	Cust_Name	Cust_Address	Product_ID	Product_Name	Price	Category_Name	Qty
			Chennai				cs	

Even in four rows, Arun's name and address appear three times, the Laptop's details appear twice, and the Mouse's details appear twice.

5. Functional Dependencies in the Flat Table

5.1 Full Functional Dependency (depends on the whole key)

- (Order_ID, Product_ID) → Quantity, Unit_Price

The quantity and the price paid are facts about a particular product within a particular order, so both parts of the key are needed.

5.2 Partial Dependencies (depend on only part of the key)

Determinant (part of key)	Dependent attributes
Order_ID	Order_Date, Order_Status, Customer_ID, Payment_Method, Payment_Status, Shipment_Status
Product_ID	Product_Name, Price, Category_ID, Supplier_ID

These attributes are about the order or the product alone, yet they are stored on every order-line row.

5.3 Transitive Dependencies (non-key determines non-key)

Chain	Meaning
Order_ID → Customer_ID → Cust_Name, Cust_Email, Cust_Mobile, Cust_Address	Customer details depend on the customer, not the order
Product_ID → Category_ID → Category_Name	Category name depends on the category, not the product
Product_ID → Supplier_ID → Supplier_Name	Supplier name depends on the supplier, not the product

6. Functional Dependencies of Each Entity (Week 2 and 3 Design)

Table	Primary key	Functional dependencies
Customer	Customer_ID	Customer_ID → Name, Email, Mobile, Address, Registration_Date. Also Email → Customer_ID and Mobile →

Table	Primary key	Functional dependencies
		Customer_ID (both are candidate keys).
Category	Category_ID	Category_ID → Category_Name, Description
Supplier	Supplier_ID	Supplier_ID → Supplier_Name, Contact_Info
Product	Product_ID	Product_ID → Product_Name, Price, Stock_Quantity, Category_ID
Orders	Order_ID	Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status
Order_Details	Order_Detail_ID	Order_Detail_ID → Order_ID, Product_ID, Quantity, Unit_Price. Also (Order_ID, Product_ID) → Quantity, Unit_Price.
Payment	Payment_ID	Payment_ID → Order_ID, Payment_Method, Payment_Date, Payment_Status
Shipment	Shipment_ID	Shipment_ID → Order_ID, Delivery_Address, Shipment_Date, Delivery_Status
Review	Review_ID	Review_ID → Customer_ID, Product_ID, Rating, Comments. Also (Customer_ID, Product_ID) → Review_ID (a customer reviews a product only once).

7. Design Issues Identified During Review

No.	Issue	Analysis	Decision
1	Missing Supplier to Product link	The entity list contains Supplier but no relationship to Product. A supplier supplies many products and a	Add a table Product_Supplier (Product_ID, Supplier_ID, Supply_Price)

No.	Issue	Analysis	Decision
		product may come from several suppliers (many-to-many).	
2	Orders.Total_Amount is derived data	Total_Amount = $\Sigma(\text{Quantity} \times \text{Unit_Price})$ from Order_Details. Storing it repeats information that can be calculated, and the two values could disagree.	Noted as redundancy; kept only as a deliberate convenience
3	Order_Details.Unit_Price vs Product.Price	Both hold prices, but Unit_Price records the price <i>at the time of purchase</i> . Without it, changing a product's price would change old order values.	Not a problem. Intentional, retained
4	Shipment.Delivery_Address vs Customer.Address	A customer may ship to an address different from the registered one.	Acceptable duplication, retained
5	Review uniqueness	Business rule: one review per customer per product, shown by the FD (Customer_ID, Product_ID) $\rightarrow$ Review_ID.	Recorded as a rule of the design

8. Redundancy Analysis

Repeated data	Where it repeats	Cause
Customer name, address, email, mobile	Every order line of that customer	Customer_ID $\rightarrow$ customer details stored on every row
Product name, price, category	Every order containing that product	Product_ID $\rightarrow$ product details stored on every row
Category name	Every product row in that category	Transitive dependency

Repeated data	Where it repeats	Cause
Supplier name	Every product row from that supplier	Transitive dependency
Order date, status, payment and shipment info	Every line item of that order	Partial dependency

Storage impact: if a customer places 50 orders with 5 items each, the address is stored $50 \times 5 = 250$ times instead of once. Across thousands of customers this wastes storage and makes the data harder to keep correct.

9. Data Anomalies

9.1 Insertion Anomaly

A new fact cannot be recorded without unrelated data.

- A **new product** (for example, "Keyboard") cannot be added until someone orders it, because Order_ID is part of the primary key and cannot be NULL.
- A **new customer** who has not yet ordered cannot be registered.
- A **new category** with no products cannot be added.

9.2 Deletion Anomaly

Deleting one fact unintentionally destroys another.

- If order 1003 is deleted (Divya's only order), **all of Divya's details are lost**.
- If the only order containing a product is deleted, **the product's name and price disappear** from the database.
- Deleting the last product of a category also deletes the category itself.

9.3 Update (Modification) Anomaly

Changing one fact requires many edits, and missing one causes inconsistency.

- Arun moves house. His address appears in 3 rows. If only 2 are updated, the database holds two conflicting addresses for the same customer.
- The Laptop price changes from 55000 to 52000. Updating only some rows leaves the table inconsistent.
- Renaming the category "Electronics" requires editing every product row in that category.

9.4 Summary of Anomalies

Anomaly	Example	Root cause
Insertion	Cannot add a product without an order	Product data mixed with order data (partial dependency)
Deletion	Deleting an order loses the customer	Customer data stored inside order rows (transitive dependency)
Update	Address changed in some rows only	Same fact stored in many rows

10. Conclusion

The analysis of the single-table design showed one full dependency, several partial dependencies on Order_ID and Product_ID, and three transitive dependency chains through Customer, Category and Supplier. These caused heavy redundancy and all three types of anomaly: insertion, deletion and update. The functional dependencies of each entity were documented, and five points were identified in the design review (Supplier to Product link, Total_Amount, Unit_Price, Delivery_Address, Review uniqueness). These findings show where the design needs improvement.